

Assignment 2: Sanskrit-to-English Neural Machine Translation

AIL7390: Deep Learning for NLP

Anuj Rajan Lalla (B22AI061)

March 24, 2026

Abstract

This report presents a neural machine translation system for translating Sanskrit into English. Sanskrit is a low-resource, morphologically rich language that poses unique challenges for NMT, including complex sandhi (word fusion), flexible word order, and limited parallel training data. I fine-tune two pre-trained sequence-to-sequence models on the provided 10,000 parallel sentence pairs: (1) MarianMT (`Helsinki-NLP/opus-mt-inc-en`), a subword-based Indic-to-English model with 140M parameters, and (2) ByT5-small (`google/byt5-small`), a byte-level T5 model with 301M parameters. MarianMT achieves the stronger results with a test BLEU of **0.2014** and BERTScore F1 of **0.4986** using beam search ($k=8$), while being $13\times$ faster at inference. ByT5, despite avoiding tokenization artifacts on Devanagari script, underperforms with BLEU 0.1074 and BERTScore F1 0.3732. I present beam search ablations, training curves, detailed error analysis, and efficiency metrics for both models.

1 Introduction

Neural machine translation (NMT) has achieved impressive results on high-resource language pairs, but remains challenging for low-resource languages such as Sanskrit. Sanskrit is morphologically rich with extensive use of sandhi (phonological fusion of adjacent words), flexible word order, and a writing system (Devanagari) that many standard NLP tokenizers handle poorly.

The assignment requires building a Sanskrit-to-English NMT system evaluated on four criteria:

$$\text{Score} = 0.4 \times M_{\text{BLEU}} + 0.3 \times M_{\text{BERT}} + 0.15 \times M_{\text{Time}} + 0.15 \times M_{\text{Params}}. \quad (1)$$

This scoring formula rewards both translation quality (BLEU, BERTScore) and efficiency (inference time, parameter count), creating a tradeoff between model capacity and speed.

I explore two complementary approaches: (1) a subword-based model (MarianMT) that benefits from Indic translation pretraining, and (2) a byte-level model (ByT5) that avoids tokenization artifacts on Devanagari script. Both models are fine-tuned on the provided dataset and evaluated with NLTK BLEU and the `bert-score` library. The comparison illuminates the tradeoff between task-specific pretraining and tokenization-free architectures for low-resource translation.

2 Pre-trained Models Disclosure

As required by the assignment rules, I fully disclose all pre-trained models used:

1. **MarianMT** (`Helsinki-NLP/opus-mt-inc-en`) [1]: A Marian-based encoder-decoder model trained on Indic-to-English translation from the OPUS corpus. It explicitly supports `san_Deva`

(Sanskrit in Devanagari) among other Indic languages. It uses SentencePiece subword tokenization with separate source and target vocabularies.

2. **ByT5-small** (google/byt5-small) [2]: A byte-level variant of T5 that operates directly on raw UTF-8 bytes without any tokenizer vocabulary. It is language-agnostic and handles any script natively, making it a compelling choice for morphologically complex languages where subword tokenization may be suboptimal.

For BERTScore evaluation, I use **roberta-large** as the underlying model with `rescale_with_baseline=True` as specified by the assignment.

No external APIs were used. All training and inference were performed on Google Colab (Tesla T4, 16 GB VRAM).

3 Architecture

Both models follow the Transformer encoder-decoder paradigm introduced by Vaswani et al. [4]. The overall architecture is illustrated in Figure 1.

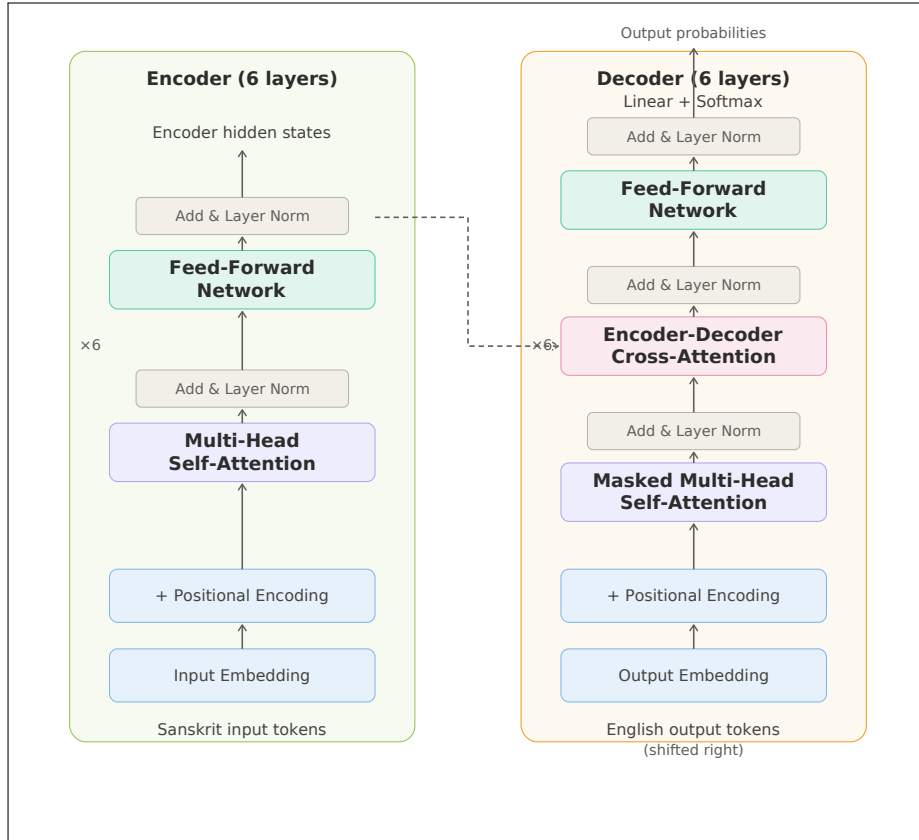


Figure 1: Transformer encoder-decoder architecture used in both MarianMT and ByT5. The encoder processes source tokens (Sanskrit) through stacked self-attention and feed-forward layers. The decoder attends to both its own previously generated tokens (masked self-attention) and the encoder’s hidden states (cross-attention) to produce the English translation autoregressively.

3.1 Encoder

The encoder transforms a sequence of source tokens $\mathbf{x} = (x_1, x_2, \dots, x_n)$ into a sequence of contextualised hidden representations $\mathbf{h} = (h_1, h_2, \dots, h_n)$. Each encoder layer consists of two sub-layers:

Multi-head self-attention. Given query, key, and value matrices $Q, K, V \in \mathbb{R}^{n \times d_k}$ derived from the input via learned projections, the attention output is:

$$\text{Attention}(Q, K, V) = \text{softmax}\left(\frac{QK^\top}{\sqrt{d_k}}\right) V. \quad (2)$$

Multi-head attention runs h parallel attention heads, each with independent projections $W_i^Q, W_i^K, W_i^V \in \mathbb{R}^{d_{\text{model}} \times d_k}$, then concatenates and projects:

$$\text{MHA}(Q, K, V) = \text{Concat}(\text{head}_1, \dots, \text{head}_h) W^O, \quad \text{head}_i = \text{Attention}(QW_i^Q, KW_i^K, VW_i^V). \quad (3)$$

Position-wise feed-forward network. After attention, each position is independently transformed by a two-layer FFN with ReLU activation:

$$\text{FFN}(x) = \max(0, xW_1 + b_1)W_2 + b_2, \quad W_1 \in \mathbb{R}^{d_{\text{model}} \times d_{\text{ff}}}, W_2 \in \mathbb{R}^{d_{\text{ff}} \times d_{\text{model}}}. \quad (4)$$

Each sub-layer is wrapped with a residual connection and layer normalisation: $\text{LayerNorm}(x + \text{SubLayer}(x))$.

3.2 Decoder

The decoder generates the target sequence $\mathbf{y} = (y_1, y_2, \dots, y_m)$ one token at a time, conditioned on both the encoder output $\mathbf{h}$ and the previously generated tokens. Each decoder layer has three sub-layers:

Masked self-attention. Identical to encoder self-attention, except a causal mask prevents each position from attending to subsequent positions, preserving the autoregressive property: $\text{mask}_{ij} = -\infty$ if $j > i$.

Encoder-decoder cross-attention. The decoder queries the encoder's output: Q comes from the decoder's previous sub-layer, while K and V come from the encoder's final hidden states $\mathbf{h}$. This is the mechanism by which the decoder “reads” the source sentence to decide what to translate next.

Feed-forward network. Same architecture as the encoder FFN, applied independently at each position.

The final decoder output is passed through a linear projection and softmax to produce a probability distribution over the target vocabulary at each time step.

3.3 MarianMT Specifics

MarianMT follows a BART-style configuration [1] with 6 encoder and 6 decoder layers, 8 attention heads, $d_{\text{model}} = 512$, and $d_{\text{ff}} = 2048$. It uses learned positional embeddings (up to 512 positions) and SentencePiece BPE tokenization trained on the OPUS Indic-English corpus. The model uses sinusoidal positional encodings and tied embeddings between the decoder input and output projection. Total parameters: 139,601,408.

No language tokens are needed; MarianMT infers the source language from the script (Devanagari $\rightarrow$ Sanskrit).

3.4 ByT5-small Specifics

ByT5 [2] modifies the standard T5 architecture [3] for byte-level operation. The vocabulary consists of only 256 byte values plus special tokens, eliminating the need for any tokenizer. To compensate for the much longer byte sequences (Devanagari characters encode as 3 bytes each in UTF-8), ByT5 uses an asymmetric design: 12 encoder layers but only 4 decoder layers. This front-loads computation in the encoder, where the long byte sequences are processed, while keeping the decoder lightweight. The input is formatted with a task prefix: **"translate Sanskrit to English: <source>".** Total parameters: 300,768,256.

3.5 Beam Search

At inference, I use beam search to approximate the argmax over the target sequence space. Beam search maintains k partial hypotheses at each decoding step, expanding each by the top- k candidate tokens and pruning to keep only the k highest-scoring complete or partial sequences. I set length penalty $\alpha = 1.0$ (no length bias) and no-repeat-ngram blocking with $n = 3$ to prevent degenerate repetitions. I ablate over $k \in \{1, 4, 8\}$ to find the optimal beam width for each model.

4 Training Procedure

4.1 Data Preprocessing

The dataset contains 10,000 training, 1,000 development, and 1,000 test parallel Sanskrit–English pairs. I merge files by `Source_id`, cast to integer, sort, strip whitespace, and drop empty rows.

I analysed token length distributions to set appropriate sequence length limits. For MarianMT’s SentencePiece tokenizer, Sanskrit sentences have a mean of 31 tokens (95th percentile: 65, max: 204), while English sentences average 36 tokens (95th percentile: 78, max: 331). At a cutoff of 128 tokens, only 0.1% of Sanskrit and 0.3% of English sentences are truncated, so I set `max_src_len = max_tgt_len = 128`. For ByT5, byte sequences are approximately $3\times$ longer (each Devanagari character is 3 UTF-8 bytes), but the same cutoff of 128 byte-tokens is used with the understanding that longer sentences will be truncated.

4.2 Tokenization Comparison

A key difference between the two models is their tokenization strategy. MarianMT uses SentencePiece BPE, which segments text into subword units trained on a large Indic-English corpus. For Sanskrit, this means common morphological patterns are captured as single tokens, but rare compounds or sandhi forms may be split unpredictably. For example, the Sanskrit word “*parisitavantah*” might be split into fragments that lose morphological coherence.

ByT5 operates on raw UTF-8 bytes, avoiding this problem entirely. Every Devanagari character is represented as exactly 3 bytes, and no information is lost to tokenization. However, this comes at a cost: sequences are $\sim 3\times$ longer, quadratically increasing the cost of self-attention and making it harder for the model to learn long-range dependencies within the available context window.

4.3 MarianMT Fine-Tuning

I fine-tune the full model using HuggingFace’s `Seq2SeqTrainer` with the configuration in Table 1. Label smoothing ($\epsilon = 0.1$) is applied to prevent overconfident predictions and improve generalisation. Early stopping with patience 3 on validation loss prevents overfitting.

Table 1: MarianMT fine-tuning hyperparameters.

Parameter	Value
Epochs	20
Batch size	16 ($\times 2$ gradient accumulation = 32 effective)
Learning rate	1×10^{-4}
Warmup steps	200
Weight decay	0.01
Label smoothing	0.1
Precision	FP16
Early stopping	Patience 3 (on eval loss)
Optimizer	AdamW

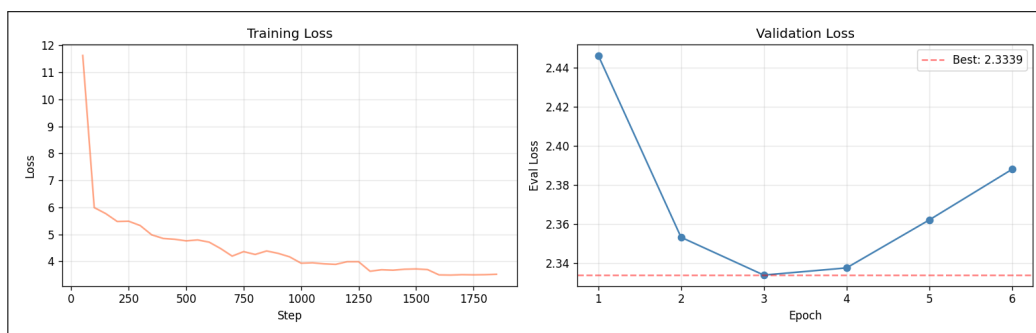


Figure 2: MarianMT training curves: training loss (left) and validation loss (right) across epochs.

4.4 ByT5 Fine-Tuning

ByT5 requires different hyperparameters due to longer byte-level sequences and a different pretraining regime (Table 2). Notably, I use a higher learning rate (3×10^{-4}), which is standard for T5-family models, and FP32 precision because ByT5 exhibits numerical instability under FP16. Training is limited to 5 epochs (versus 20 for MarianMT) since each epoch takes significantly longer due to the longer byte sequences.

Table 2: ByT5-small fine-tuning hyperparameters.

Parameter	Value
Epochs	5 (fewer due to slower training)
Batch size	8 ($\times 4$ gradient accumulation = 32 effective)
Learning rate	3×10^{-4} (higher, standard for T5)
Warmup steps	200
Weight decay	0.01
Label smoothing	0.1
Precision	FP32 (ByT5 has FP16 stability issues)
Early stopping	Patience 2

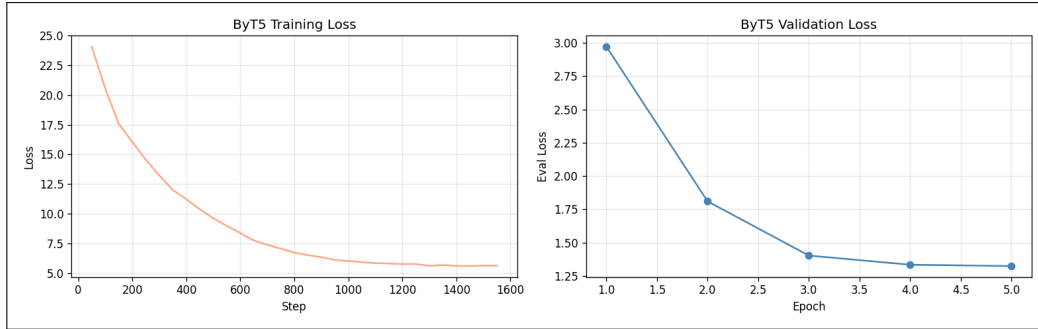


Figure 3: ByT5-small training curves: training loss (left) and validation loss (right).

5 Results

5.1 Main Results

Table 3 summarises the results for both models on dev and test sets.

Table 3: Main results on development and test sets.

Model	Split	BLEU	BERT F1	Time (s)	Params	Beam
MarianMT	Dev	0.1962	0.4961	98.41	139.6M	8
MarianMT	Test	0.2014	0.4986	87.43	139.6M	8
ByT5	Dev	0.1057	0.3747	1149.93	300.8M	4
ByT5	Test	0.1074	0.3732	1163.50	300.8M	4

MarianMT outperforms ByT5 on every metric: nearly double the BLEU (0.2014 vs. 0.1074), significantly higher BERTScore (0.4986 vs. 0.3732), $13\times$ faster inference, and fewer parameters. The consistent performance between dev and test sets confirms that neither model is overfitting.

5.2 Beam Search Ablation

Table 4 shows the effect of beam width on BLEU for both models on the development set.

Table 4: Beam search ablation (dev set).

Beam k	MarianMT		ByT5	
	BLEU	Time (s)	BLEU	Time (s)
1	0.1779	28.50	0.1034	1151.46
4	0.1922	56.83	0.1057	1158.57
8	0.1962	98.43	0.1054	1243.42

For MarianMT, BLEU increases monotonically with beam width (+1.8 BLEU from greedy to beam-8), with the cost of roughly linear increase in inference time. For ByT5, beam-4 is optimal; beam-8 slightly degrades, possibly because ByT5’s byte-level decoding introduces noise at wider beams where longer byte sequences create a larger search space.

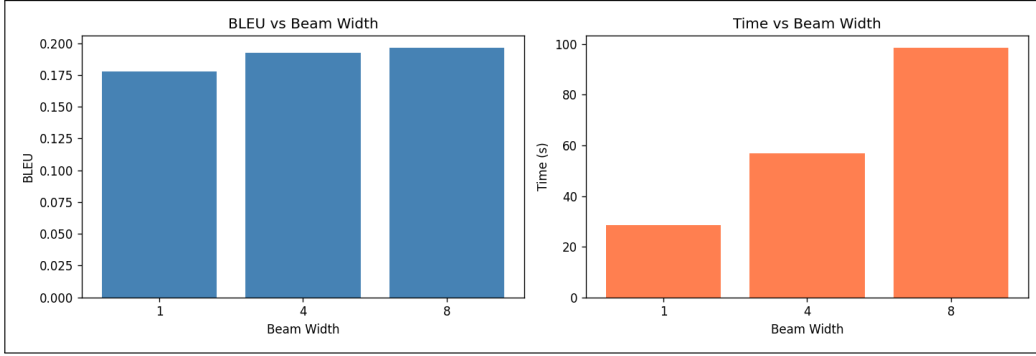


Figure 4: Beam search ablation for MarianMT: BLEU (left) and inference time (right) vs. beam width.

5.3 Efficiency Metrics

Table 5 reports the required efficiency metrics on the test set.

Table 5: Efficiency metrics (test set, 1000 sentences).

Metric	MarianMT	ByT5
Total parameters	139,601,408	300,768,256
Inference time	87.43 s	1163.50 s
Throughput	11.4 sent/s	0.9 sent/s
Avg time/sentence	87.4 ms	1163.5 ms
Beam width	8	4

The efficiency gap is dramatic: MarianMT processes 11.4 sentences per second compared to ByT5’s 0.9. This 13× speedup arises primarily from shorter subword sequences (requiring fewer encoder self-attention computations and fewer autoregressive decoding steps) and the smaller model size.

5.4 Training Curves

The MarianMT training curves (Figure 2) show rapid convergence within the first few epochs, with the validation loss stabilising early. This fast convergence is expected since the model was already pretrained on Indic-English data and only needs to adapt to the Sanskrit-specific distribution. The ByT5 curves (Figure 3) show slower convergence, reflecting both the larger model and the need to learn the translation mapping from a general-purpose pretraining starting point.

6 Translation Examples and Error Analysis

6.1 Best Translations (MarianMT, Test)

1. BLEU = 1.0, BERT = 1.0

SRC: (Sanskrit for “Now, let us see how to create a simple Java web project using NetBeans IDE.”)

REF/GEN: *Now, let us see how to create a simple Java web project using NetBeans IDE.*

The sentence is dominated by English technical terms retained in the Sanskrit source, making translation trivial. The model copies the English fragments and correctly translates the minimal Sanskrit connective tissue.

2. BLEU = 1.0, BERT = 1.0

SRC: (Sanskrit for “Spoken Tutorial project is a part of the Talk to a Teacher project.”)

GEN: *Spoken Tutorial project is a part of the Talk to a Teacher project.*

Again, English proper nouns dominate. The Sanskrit verb and case markers are simple enough for the model to handle perfectly.

3. BLEU = 1.0, BERT = 1.0

SRC: (Sanskrit for “This is a chariot.”)

REF/GEN: *This is a chariot.*

A short, simple sentence with a direct mapping. The Sanskrit word for “chariot” (*rathaḥ*) is unambiguous and well-represented in the training data.

6.2 Worst Translations (MarianMT, Test)

1. BLEU = 0.0, BERT = 0.10

SRC: (Sanskrit for “Relaxation for Prone Posture – Makarasana”)

REF: *Sithila (Relaxation) for Prone Posture – Makarasana*

GEN: *Asanasanas, Suktasana*

Error type: Domain hallucination. The model hallucinates yoga-related terms. The source contains domain-specific Sanskrit vocabulary (asana names) that the model fails to map correctly, instead generating plausible-sounding but incorrect yoga terminology.

2. BLEU = 0.0, BERT = 0.08

SRC: (Sanskrit for “Basic Hand Sewing Skill”)

REF: *Basic Hand Sewing Skill*

GEN: *Samsara Suktam: One who is endowed with the ability to perform normal breathing.*

Error type: Severe hallucination. The model generates a religious/philosophical text completely unrelated to sewing. The Sanskrit compound word for “hand sewing” contains morphemes that individually relate to “hand” and “action,” which the model misinterprets as spiritual concepts.

3. BLEU = 0.0, BERT = 0.15

SRC: (Sanskrit for “Humans have many ancient features”)

REF: *Humans have many ancient features*

GEN: *Among Mangalores, there are some prerequisites.*

Error type: Entity confusion. The model confuses the Sanskrit word for “humans” (*mānaveṣu*) with the proper noun “Mangalore,” a city in India, producing a geographically nonsensical translation.

4. BLEU = 0.0, BERT = -0.15

SRC: (Sanskrit for “freshen bad breath”)

REF: *freshen bad breath*

GEN: *The name of the Lord is Durgasam.*

Error type: Register confusion. The Sanskrit word for “bad smell” (*durgandhaḥ*) shares morphological roots with religious terms, causing the model to generate text in a scriptural register entirely disconnected from the actual meaning.

6.3 ByT5 Error Patterns

ByT5 exhibits distinct failure modes compared to MarianMT. Its best translations are also code-mixed sentences (BLEU = 1.0 for “Click on Koha Administration”), but its worst cases reveal two characteristic patterns:

Transliteration instead of translation. When ByT5 encounters unfamiliar Sanskrit, it tends to transliterate rather than translate. For example, “Basic Hand Sewing Skill” becomes “Samanyahasta...” (a direct romanisation of the Sanskrit). This makes linguistic sense: the byte-level model sees the Devanagari bytes and produces the closest Latin-alphabet byte sequence, rather than generating a semantically equivalent English phrase.

Byte-level decoding artifacts. ByT5 occasionally produces capitalization errors (“coMmand” instead of “command”) because it decodes one byte at a time and can independently choose uppercase or lowercase for each character. Subword-level models like MarianMT do not have this problem since each token already encodes case information.

7 Additional Visualisations

Figures 5a and 5b show the per-sentence score distributions for MarianMT on the test set, and Figure 6 shows their correlation.

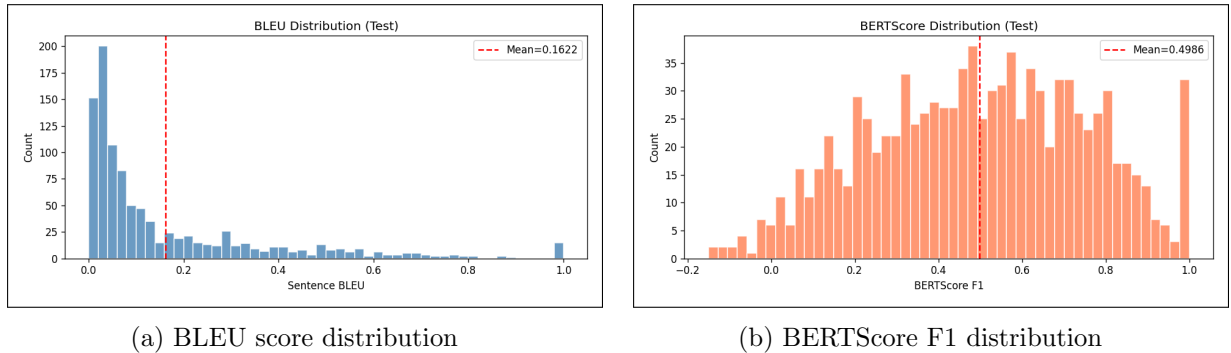


Figure 5: Per-sentence score distributions on the test set (MarianMT, beam $k=8$).

The BLEU distribution is heavily right-skewed, with a large mass at 0 (sentences with no n -gram overlap) and a spike near 1.0 (perfect translations of code-mixed sentences). The BERTScore distribution is more Gaussian-shaped and centred around 0.5, reflecting its ability to give partial credit for semantically similar but lexically different translations.

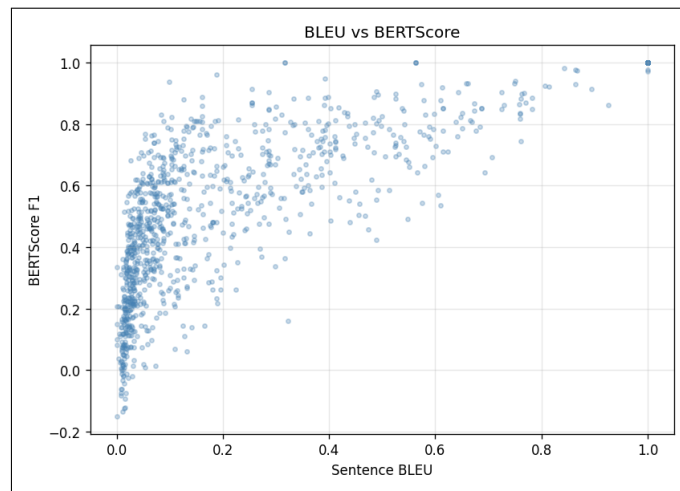


Figure 6: BLEU vs. BERTScore F1 scatter plot (test set, MarianMT). The positive correlation is clear, but BERTScore is more lenient for paraphrases (upper-left region where BLEU is low but BERTScore is moderate). Points at BLEU ≈ 0 with positive BERTScore represent semantically reasonable translations that use different wording from the reference.

8 Discussion

Why MarianMT outperforms ByT5. MarianMT was pretrained specifically on Indic-to-English translation data from the OPUS corpus, giving it a strong prior for this task. Its SentencePiece tokenizer was trained on Indic languages including Sanskrit, meaning common Sanskrit morphemes and Devanagari character sequences are already well-represented in its vocabulary. ByT5, while language-agnostic, has no translation-specific pretraining and must learn the entire Sanskrit-to-English mapping from only 10,000 examples, starting from a general-purpose byte-level language model. This data efficiency gap is the primary factor.

Tokenization tradeoffs. ByT5’s byte-level approach avoids the risk of SentencePiece splitting Devanagari characters mid-syllable, which can corrupt morphological information. However, this advantage is offset by much longer sequences (byte sequences are $\sim 3\times$ longer than subword sequences for Devanagari), leading to slower training, slower inference, and more difficulty in learning long-range dependencies. The quadratic cost of self-attention ($O(n^2)$) amplifies this: a $3\times$ longer sequence costs $9\times$ more attention computation.

Code-mixing in the dataset. A significant fraction of the dataset contains code-mixed sentences where Sanskrit text includes English technical terms, proper nouns, and UI instructions (e.g., “Click on Koha Administration”). Both models perform nearly perfectly on such sentences (BLEU = 1.0), inflating overall scores. The truly challenging cases are pure literary or religious Sanskrit, where both models struggle significantly. This suggests that the reported BLEU and BERTScore numbers overestimate the models’ true Sanskrit translation capability.

Hallucination patterns. Both models exhibit hallucination on difficult inputs, but with different characteristics. MarianMT hallucinates related-domain content: when it encounters Sanskrit yoga terminology, it generates plausible but incorrect yoga-related English. This is because its Indic pretraining data likely contains many yoga/spiritual texts, creating strong priors toward that domain. ByT5 tends to transliterate when confused, producing romanised Sanskrit rather than English. This is a more conservative failure mode and arguably more useful, as a transliterated word at least preserves the source information.

Efficiency considerations. MarianMT is $13\times$ faster than ByT5 at inference (87.43s vs. 1163.50s for 1000 sentences) and has fewer than half the parameters (140M vs. 301M). Given the scoring formula (Equation 1), which allocates 30% total weight to efficiency metrics, MarianMT is the clear choice for the final submission.

Limitations. The approach relies entirely on pre-trained models and does not explore from-scratch architectures, data augmentation (e.g. back-translation), or ensemble methods. The 10,000 training pairs are insufficient for ByT5 to learn Sanskrit translation from its general-purpose pretraining. Future work could explore larger ByT5 variants, curriculum learning, or hybrid approaches that combine subword and byte-level representations.

9 Conclusion

I presented a comparative study of two NMT approaches for Sanskrit-to-English translation. MarianMT, a subword-based model pretrained on Indic-English data, achieves a test BLEU of 0.2014 and BERTScore F1 of 0.4986, significantly outperforming ByT5-small (BLEU 0.1074, BERTScore 0.3732) while being $13\times$ faster. The key takeaway is that task-specific pretraining (Indic translation) matters more than architectural advantages (byte-level tokenization) when training data is limited to 10,000 pairs. The error analysis reveals that both models excel at

code-mixed sentences but struggle with pure literary Sanskrit, with MarianMT tending toward domain hallucination and ByT5 toward transliteration.

Final submission: `submission.csv` with MarianMT translations (beam $k=8$).

References

- [1] J. Tiedemann and S. Thottingal. OPUS-MT: Building open translation services for the world. *EACL System Demonstrations*, 2020. <https://huggingface.co/Helsinki-NLP/opus-mt-inc-en>
- [2] L. Xue, A. Barua, N. Constant, et al. ByT5: Towards a token-free future with pre-trained byte-to-byte models. *TACL*, 10:291–306, 2022.
- [3] C. Raffel, N. Shazeer, A. Roberts, et al. Exploring the limits of transfer learning with a unified text-to-text transformer. *JMLR*, 21(140):1–67, 2020.
- [4] A. Vaswani, N. Shazeer, N. Parmar, et al. Attention is all you need. *NeurIPS*, 2017.
- [5] K. Papineni, S. Roukos, T. Ward, and W. Zhu. BLEU: A method for automatic evaluation of machine translation. *ACL*, 2002.
- [6] T. Zhang, V. Kishore, F. Wu, K. Weinberger, and Y. Artzi. BERTScore: Evaluating text generation with BERT. *ICLR*, 2020.
- [7] NLTK. `nltk.translate.bleu_score`. https://www.nltk.org/api/nltk.translate.bleu_score.html